1 Függvénykönyvtár mátrixműveletekhez

A függvénykönyvtár célja, hogy a Python kiegészítő függvénykönyvtárainak (pl. numpy) használata nélkül tudjak mátrixszámításokat végezni és hogy képes legyen bármilyen nagy mátrixszal dolgozni, fájlba írni és visszaolvasni azt.

A mátrix függvénykönyvtár az alábbi függvényeket valósítja meg:

1. Mátrix létrehozása listából
2. Mátrix létrehozása fájlból
3. Mátrix kiírása fájlba
4. Mátrix összeadása mátrixszal
5. Mátrix kivonása mátrixból
6. Mátrix szorzása számmal
7. Mátrix szorzása mátrixszal
8. Mátrix osztása számmal
9. Mátrix determinánsa
10. Mátrix inverze
11. Mátrix rangja
12. Mátrix transzponáltja
13. Gauss-elimináció

2 A függvénykönyvtár használata

A függvénykönyvtárt a szokásos módon kell telepíteni.

```
pip install -e <könyvtár>
```

A <könyvtár> annak a könyvtárnak az elérési útvonala, ahol a 'setup.py' található.

A függvénykönyvtár egy mátrix típust definiál. A mátrix egy n hosszúságú listát tartalmaz, ahol a lista minden eleme egy m hosszúságú számokat tartalmazó lista. A továbbiakban a mátrix osztályt `<Matrix>` módon nevezzük.

3 Adatszerkezet:

A mátrixot egy listában tároljuk, ami n darab azonos hosszúságú listát tartalmaz.

A `<matrixA.matrix_sor>` a `<matrixA>` mátrix sorainak számát adja meg. A `<matrixA.matrix_oszlop>` a `<mátrixA>` oszlopainak számát adja meg.

Amennyiben a mátrix 0 elemű a kód hibát dob.

```
class Matrix:
    adat = []
    matrix_sor = 0
    matrix_oszlop = 0
    def __init__(self,lista):
        self.adat = lista
        self.matrix_sor = len(lista)
        if self.matrix_sor < 1:
            raise ValueError("A mátrix nem lehet 0 elemű.")
        self.matrix_oszlop = len(lista[0])
        for sor in lista:
```

```
            hosz = len(sor)
            if hosz != self.matrix_oszlop:
                raise ValueError("A mátrix minden sorának azonos hosszúnak
kell lennie.")
```

4 A program működését vezérlő fő osztályok és függvények:

4.1 Class Matrix

Egy mátrix adatait tárolja: A mátrix adata (egy listában n darab azonos hosszú lista): `<Matrix>.adat`. A mátrix sorainak száma: `<Matrix>.matrix_sor.` A mátrix oszlopainak száma: `<Matrix>.matrix_oszlop.`

4.2 Mátrix létrehozása listából

A függvény létrehoz egy <Matrix> objektumot egy listából. A lista n darab azonos hosszúságú listákat tartalmaz, melyek a mátrix sorait jelenítik meg.

A függvényt az alábbi módon lehet meghívni:

```
<eredmeny> = Matrix(<lista>)
```

Az <eredmeny> egy <Matrix> típus. A lista egy n sort és m oszlopot tartalmazó lista.

4.3 Mátrix létrehozása fájlból

A függvény beolvassa a fájlban lévő mátrixot.

A fájl egy szövegfájl, és csak egy mátrix adatait tartalmazhatja.

A fájlban egy sor a mátrix egy sorát jelenti. Az mátrix egyes elemei ” , ” jellel vannak elválasztva.

Ha a fájlban nem ugyanakkorák a sorok a függvény hibát dob.

Minta állomány:

```
1,2,3.5,6
2,3,4,5.6
2,3,3,-7.8
```

4.4 Mátrix kiírása fájlba

A függvény megnyit egy fájlt és beleírja egy mátrix adatait. A fájl egy szövegfájl, és csak egy mátrix adatait tartalmazza.

A fájlban egy sor a mátrix egy sorát jelenti. Az mátrix egyes elemei ” , ” jellel vannak elválasztva.

4.5 Mátrix összeadása mátrixszal

A függvényt két mátrix összegét adja meg egy új mátrixként.

A mátrix összeadás az alábbi módon történik: legyen A és B két azonos dimenziójú, n*m-es méretű mátrix. Az A+B összeget úgy képezzük, hogy az azonos helyen lévő elemeket összegezzük.

A függvényt az alábbi módon lehet meghívni:

```
<eredmeny> = matrix_osszeadas(<matrixA>, <matrixB>)
```

Az <eredmeny>, <matrixA>, <matrixB> is n*m dimenzójú <Matrix> típusú.

A függvények minden argumentumát kötelező kitölteni!

Ha a két mátrix nem azonos dimenziójú a függvény hibát dob.

4.6 Mátrix kivonása mátrixból

A függvényt két mátrix különbségét adja meg egy új mátrixként.

A mátrix kivonás az alábbi módon történik: Legyen A és B két azonos dimenziójú, n*m-es méretű mátrix. Az A-B különbséget úgy képezzük, hogy az azonos helyen lévő elemek különbségét vesszük úgy, hogy az A mátrix a kisebbítendő a B mátrix a kivonandó.

A függvényt az alábbi módon lehet meghívni:

```
<eredmeny> = matrix_kivonas(<matrixA>, <matrixB>)
```

Az <eredmeny>, <matrixA>, <matrixB> is n*m dimenziójú <Matrix> típusú.

A függvények minden argumentumát kötelező kitölteni!

Ha a két mátrix nem azonos dimenziójú a függvény hibát dob.

4.7 Mátrix szorzása számmal

A függvény egy mátrix és egy szám szorzatát adja eredményül.

A mátrix szorzás számmal az alábbi módon történik: Legyen A egy n*m-es mátrix és sz egy szám. Az A*sz szorzatot úgy képezzük, hogy az A mátrix minden elemét megszorozzuk az sz számmal.

A függvényt az alábbi módon lehet meghívni:

```
<eredmeny> = matrix_szorzas_szammal(<matrixA>, <sz>)
```

Az <eredmeny>, <matrixA> is n*m dimenziójú <Matrix> típusú.

A függvények minden argumentumát kötelező kitölteni!

4.8 Mátrix szorzása mátrixszal

A függvény két mátrix szorzatát adja eredményül.
A mátrix szorzás mátrixszal az alábbi módon történik: Legyen A egy n*m-es, B egy m*n-es méretű mátrix. Az <eredmeny> i-edik sorának j-edik elemét úgy kapjuk, hogy az A mátrix i-edik sorát skalárisan összeszorozzuk a B mátrix j-edik oszlopával.

A függvényt az alábbi módon lehet meghívni:

```
<eredmeny> = matrix_szorzas_matrixszal(<matrixA>, <matrixB>)
```

Az <eredmeny> m*m dimenziójú a `<matrixA>` és `<matrixB>` n*m dimenziójú <Matrix> típusú.

A függvények minden argumentumát kötelező kitölteni!

4.9 Mátrix osztása számmal

A függvény egy mátrix és egy szám hányadosát adja eredményül.

A mátrix osztása számmal az alábbi módon történik: Legyen A egy n*m-es méretű mátrix és sz egy szám. Az A/sz hányadost úgy kapjuk, hogy az A mátrix összes elemét elosztjuk az sz számmal.

A függvényt az alábbi módon lehet meghívni:

```
<eredmeny> = matrix_osztasa(<matrixA>, <sz>)
```

Az <eredmeny>, <matrixA> n*m dimenziójú <Matrix> típusú. Az <sz> egy tetszőleges szám.

A függvények minden argumentumát kötelező kitölteni!

4.10 Mátrix determinánsa

A függvény egy mátrix determinánsát adja eredményül.

A mátrix determinánsának kiszámítása az alábbi módon működik:

2×2-es mátrix determinánsa:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \qquad det(A) = ad - bc$$

3×3-as mátrix determinánsa:

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \qquad \det(A) = a\begin{vmatrix} e & f \\ h & i \end{vmatrix} - b\begin{vmatrix} d & f \\ g & i \end{vmatrix} + c\begin{vmatrix} d & e \\ g & h \end{vmatrix}$$

n*n-es mátrix determinánsa i. sor szerinti kifejtéssel:

Jelölje: A_{ij} az i. sor és a j. oszlop elhagyásával keletkező minormátrixot! A minormátrixok determinánsát aldeterminánsnak nevezzük. A kifejtés során minden tagban a $a_{ij}\det(A_{ij})$ kifejezést meg kell szorozni $(-1)^{i+j}$-vel. Ekkor a determináns az i. sor szerinti kifejtést használva[i]:

$$\det(A) = \sum_{j=1}^{n} (-1)^{i+j} a_{ij} \det(A_{ij})$$

A függvényt az alábbi módon lehet meghívni:

```
<eredmeny> = matrix_determinansa(<matrixA>)
```

A <matrixA> egy n*n-es méretű <Matrix> típusú. Az <eredmeny> egy szám.

Determinánsa csak négyzetes(n*n-es) mátrixoknak van.

A függvények minden argumentumát kötelező kitölteni!

Feltételben megnézzük, hogy a mátrixnak lehet-e determinánsa.

```
if matrixA.matrix_sor != matrixA.matrix_oszlop:
        raise ValueError("Csak n*n es mátrixoknak van determinánsa.")
```

Ha a mátrixnak egy eleme van akkor a determinánsa önmaga.

```
if len(matrixA.adat[0]) == 1:
        return matrixA.adat[0][0]
```

A 2*2-es mátrixot képlettel számítjuk ki.

```
elif len(matrixA.adat) == 2:
        return matrixA.adat[0][0] * matrixA.adat[1][1] - matrixA.adat[0][1] *
matrixA.adat[1][0]
```

Ha nem 2*2-es a mátrix akkor egy rekurzív képlettel számoljuk ki.

```
        det = 0
        for i in range(len(matrixA.adat)):
            elojel = (-1) ** i
            almatrix = []
            for sor in matrixA.adat[1:]:
                almatrix.append(sor[:i] + sor[i + 1:])
            matrix = Matrix(almatrix)
            aldet = matrix_determinansa(matrix)
            det += elojel * matrixA.adat[0][i] * aldet
```

4.11 Mátrix inverze

A függvény egy mátrix inverzét adja vissza eredményül.

Egy mátrixnak akkor van inverze, ha a determinánsa nem nulla.

A mátrix inverzének kiszámítása az alábbi módon történik: Legyen A egy n×n-es méretű mátrix. Az A mátrix inverze az A^{-1} n×n-es méretű mátrix.

$$AA^{-1} = A^{-1}A = E$$

Ahol A^{-1} az A mátrix inverze, és E az A mátrix egységmátrixa.

A függvényt az alábbi módon lehet meghívni:

`<eredmeny> = matrix_inverze(<matrixA>)`

Az <eredmeny> és a <matrixA> is n*n dimenziójú <Matrix> típusú.

A függvények minden argumentumát kötelező kitölteni!

Feltételben megnézzük, hogy nulla-e a mátrix determinánsa, ha igen, akkor a mátrixnak nem létezik inverze.

```
    det = matrix_determinansa(matrixA)
    if det == 0:
        return "A mátrixnak nem létezik inverze"
```

Létrehozunk egy egységmátrixot két `for` ciklus segítségével.

```
    sorok = matrixA.matrix_sor
    egysegmatrix = []
    for i in range(len(matrixA.adat)):
        sor = []
        for j in range(len(matrixA.adat)):
            if i == j:
                sor.append(1)
            else:
                sor.append(0)
        egysegmatrix.append(sor)
```

Kibővítjük a mátrixot az egységmátrixszal.

```
    for sor in range(sorok):
        matrixA.adat[sor] += egysegmatrix[sor]
```

Normáljuk a mátrix adott sorát.

```
    for sor in range(sorok):
        oszlop = sor
        oszto = matrixA.adat[sor][sor]

        for j in range(sorok*2):
            matrixA.adat[sor][j] /= oszto
```

A mátrixot Gauss-Jordan módszerrel alakítjuk több egymásba ágyazott for ciklus használatával.

```
        for i in range(sorok):
            if i != sor:
                szorzo = matrixA.adat[i][oszlop]
                for j in range(sorok*2):
                    matrixA.adat[i][j] -= szorzo * matrixA.adat[sor][j]
```

4.12 Mátrix rangja

A függvény egy mátrix rangját adja eredményül.

A mátrix rangjának kiszámítása az alábbi módon történik: Legyen A egy n*m-es mátrix. Az A mátrix rangját úgy kapjuk, hogy a mátrixot átalakítjuk Gauss-eliminációval, hogy a főátló alatt csak 0-ák legyenek. A nem csak nulla sorok száma a mátrix rangja.

A függvényt az alábbi módon lehet meghívni:

`<eredmeny> = matrix_rangja(<matrixA>)`

Az <eredmeny> egy szám, a <matrixA> egy n*m dimenziójú <Matrix> típus.

A függvények minden argumentumát kötelező kitölteni!

4.13 Mátrix transzponáltja

A függvény egy mátrix transzponáltját adja eredményül.

A mátrix transzponáltjának a kiszámítása az alábbi módon történik: legyen A egy n*m-es méretű mátrix. A mátrix transzponálása a mátrix sorainak és oszlopainak felcserélése az alábbi módon: Az A mátrix első oszlopát felcseréljük az első sorával, majd a második oszlopát az második sorával és így tovább[ii].

A függvényt az alábbi módon lehet meghívni:

```
<eredmeny> = matrix_transzponaltja(<matrixA>)
```

Az <eredmeny> m*n-es méretű a <matrixA> n*m-es méretű <Matrix> típusú.

A függvények minden argumentumát kötelező kitölteni!

4.14 Gauss-elimináció

A függvény egy mátrix Gauss-eliminált (jelen esetben egy n*n+1-es méretű mátrixot) adja eredményül.

A Gauss-elimináció a lineáris algebra egy lineáris egyenletrendszerek megoldására használatos algoritmusa[iii].

A mátrix Gauss-eliminált az alábbi módon számíthatjuk ki: Legyen A egy n*n+1-es méretű mátrix, amit egy n*n ismeretlenes egyenletrendszerből képzünk. Ezt a mátrixot kell átalakítani felsőháromszög mátrixszá az alábbi lépéseket használva:

- egy sor szorzása konstanssal
- két sor megcserélése
- egy sor konstansszorosának kivonása egy másik sorból
- egy sor konstansszorosának hozzáadása egy másik sorhoz

A függvényt az alábbi módon lehet meghívni:

```
<eredmeny> = matrix_Gauss-eliminacio(<matrixA>)
```

Az <eredmeny> és <matrixA> is n*n+1-es méretű <Matrix> típusú.

A függvények minden argumentumát kötelező kitölteni!

---

[i] https://hu.wikibooks.org/wiki/Matematika/M%C3%A1trix/Determin%C3%A1ns
[ii] https://hu.wiktionary.org/wiki/m%C3%A1trix_transzpon%C3%A1ltja
[iii] https://hu.wikipedia.org/wiki/Gauss-elimin%C3%A1ci%C3%B3